

Branching

If-else Branches

- There is a fork in the road. What do you do?
 - Turn Left?
 - Turn Right?

If-else Branches

- Pick it up!

If-else Branches

- Each day we make decisions based on our given context.
 - Are we there yet?
 - Can you ride this ride?
 - Are you taller than 48"?

If-else Branches

- These are examples of If-else statements
 - Two parts
 - if statement: is the requirement met?
 - if yes, then this block of code executes
 - else statement
 - executes if the previous conditions are not met, then this will execute

```
value = int(input("Please enter a number between 0 and 100"))

if value > 10:
    print(value, 'is greater than 10')
else:
    print(value, 'is less than or equal to 10')
```

If-else Branches

- If statements do not require else.
- else statements require at least an if

Conditional Operators

- **Conditional operators** assist us with determining whether or not a condition has been met.
- Conditional operators used in a statement will return a Boolean as True or False.
 - True means the condition is met
 - False means the condition is not met

Operator	Example	Purpose
>	<code>x > y</code>	Evaluates as True if x has a value greater than y
<	<code>x < y</code>	Evaluates as True if x has a value that is less than y
>=	<code>x >= y</code>	Evaluates as True if x is greater than or equal to the value of y
<=	<code>x <= y</code>	Evaluates as True if x is less than or equal to the value of y
==	<code>x == y</code>	Evaluates as True if x has the same value as y
!=	<code>x != y</code>	Evaluates as True if x is not the same value as y

Logical Operators

- When handling multiple conditions, we may choose to use **logical operators**.

Operator	Example	Purpose
and	<code>x > y and z == 3</code>	If both conditions to the left and right of the and are True, this will be evaluated as True. Otherwise it will be False
or	<code>x > y or z == 3</code>	If at least one of the conditions are True, this will evaluate as True. It will evaluate as False if neither condition is met.
not	<code>not x > y</code>	If the condition proceeding the not evaluates as False, then the statement is evaluated as True. Think of it as providing the opposite evaluation.

Logical Operators

- True and True -> True
- True and False -> False
- False and True -> False
- False and False -> False
- True or True -> True
- True or False -> True
- False or True -> True
- False or False -> False

Logical Operators

- not True and True -> False
 - not True and False -> True
 - not False and True -> True
 - not False and False -> True
- not True or True -> True
 - not True or False -> False
 - not False or True -> True
 - not False or False -> True

Expressions

- Conditional expressions are used to evaluate if certain conditions are met. These can use both conditional operators and logical operators.
- These can be used to evaluate branching statements, or set a variable to a Boolean based on conditions.

Multiple Conditions

- Multiple outcomes can occur using another branching statement - elif

```
if condition_1:  
    # results for condition 1  
elif condition_2:  
    # results for condition 2  
else:  
    # results for all other conditions
```

Multiple Conditions

- If the first condition is not met, then it evaluates the second condition.
- Multiple elifs are allowed

Ranges

- when evaluating a value, you can evaluate a range.
- Two common ways
- Example: If the value of x is between 10 and 15
 - if $x \geq 10$ and $x \leq 15$:
 - if $10 \leq x \leq 15$:
 - Both are viable options

Ranges

- How would we do the opposite?
 - if $x < 10$ or $x > 15$:
 - if not $10 \leq x \leq 15$:
- Nesting is also an option, but not necessarily preferred

Ranges

```
if x >= 10:
    if x <= 15:
        # result
    else:
        # result
else:
    # result
```

Conditional Expressions

- `value = x if condition else y`
- The above assigns `x` to `value` if condition is `True`, otherwise `value` is assigned `y`
- Also referred to as a **ternary expression**

Order of Evaluation

- When utilizing conditional and logical operators be aware of precedence rules.
- Evaluation is done as follows

Operator	Description	Explanation
()	Items within parentheses are evaluated first	In $(a - (b + c)) - d$, the $+$ is evaluated first, then $-$, then $-$.
$*, /, \%, +, -$	Arithmetic operators (using their precedence rules)	$z - 45 - y < 53$ evaluates $-$, then $-$, then $<$
$<, <=, >, >=, ==, !=$	Relational, (in)equality, and membership operators	$x < 2$ or $x >= 10$ is evaluated as $(x < 2)$ or $(x >= 10)$ because $<$ and $>=$ have precedence over or
not	not (logical NOT)	not x or y is evaluated as $(not\ x)$ or y .
and	Logical AND	$x == 5$ or $y == 10$ and $z != 10$ is evaluated as $(x == 5)$ or $((y == 10) and\ z\ (z != 10))$ because and has precedence over or
or	Logical OR	$x == 7$ or $z < 2$ is evaluated as $(x == 7)$ or $(x < 2)$ because $<$ and $==$ have precedence over or

Common Mistakes

- using `=` instead of `==` when evaluating for equality
- creating an else statement with a condition (i.e. `else x < y:`)
- Using multiple ifs instead of a sequence of if-elif-else

Common Mistakes

```
if condition1:  
    # code  
if condition2:  
    # code  
else:  
    # code
```

Common Mistakes

- The code could execute both if statements because the first if condition is standalone

```
if condition1:  
    # code  
if condition2:  
    # code  
else:  
    # code
```

Common Mistakes

- Accidental gaps in ranges
- Condition that is too broad or not specific enough

```
x = 10

if x > 5:
    # code
elif x > 8:
    # code
else:
    # code
```

Common Mistakes

- Accidental gaps in ranges
- Condition that is too broad or not specific enough
- the elif will never be reached in this instance

```
x = 10

if x > 5:
    # code
elif x > 8:
    # code
else:
    # code
```